Bits, Bytes, and Data Representation: A single bit is 0 or 1, and 8 bits make a byte.Hex: 4 bits = 1 hex digit. An 8-bit unsigned integer ranges 0–255.Negative integers are commonly stored in two's complement.)ASCII: 7-bit, 128 chars, English only.Unicode: 32-bit code space, 140k chars, 150+ langs, includes emoji.programs and data both reside in memory as binary. OS: privileged program controlling hardware, providing services: process, memory, files system, I/O, network, security (user vs system mode, syscalls). File system: tree of directories and files, root "/", pathnames like:/MyFiles/INFO1112/lecture1.pdf. Common FS: ext4 (Linux), NTFS (Windows)

Shell and Processes: a command interpreter, the shell is a program that anyone could write NAME=value (no spaces around =), and retrieve with $NAME The condition [ ] uses the test command. Note the spaces and the use of == for string comparison in bash

cd	Change directory	..	Parent	`cd ..`
mkdir	Make directories	.	Current	`./program`
ls	List directory	-	Previous	`cd -`
rmdir	Remove directory	~	Home	`cd ~`
rm	Remove	/	Root	`cd /`
mv	Move(rename)	cd	Copy	
pwd	Print work Directory			

FD	0: `stdin`, 1: `stdout`, 2: `stderr`
`>file`	same as `1>file`: stdout overwrite
`>>file`	same as `1>>file`: stdout append
`2>file`	`stderr` overwrite
`2>>file`	`stderr` append
`N>file`	redirect fd `N`
`echo "Hazem" > 2`	create / overwrite file 2
`echo "Err" >&2`	stdout → stderr
`cmd >file 2>&1`	stdout → `file`, stderr → same as stdout (file)
`cmd1 | cmd2`	stdout(cmd1) → stdin(cmd2)

Common Shell Commands:

`cat file`	Print document content
`head file`	View the first few lines of the file
`tail file`	View the last few lines of the file
`grep "pattern" file`	Match lines by regular expression / pattern and output
`wc file`	Count lines, words, and characters
`man cmd`	View the help documentation for `cmd`

A process = a running program + its state, Starting a program in the shell = creating a new process.

`fork()`	The child has a new PID; it returns 0 in the child list and returns the child's PID in the parent list.
`exec()`	replace process with new program, PID unchanged, Original disappears,New executes from the beginning
`wait()`	child exits after execution, parent returns from the wait state, prevents zombie processes

Backend job:View: jobs,Switch to foreground: fg , Pause: Ctrl+Z (SIGTSTP), Terminate: Ctrl+C (SIGINT), A process may be in the following state: running, ready, waiting / blocked, terminated. In Unix, every process has a parent process (PPID): shell → fork → child → exec → child exit → parent wait

UID / Permissions	
UID	user ID, controls which files a process can access
GID	group ID
UID = 0	`root`, superuser privileges
shell	runs commands with the current user's UID

Viewing processes (ps)	
PID	process ID
PPID	parent process ID (for parent–child relation)
CMD	command / program name
STAT	process state
TTY	terminal / controlling tty

Emulators, Virtual Machines, and Containers: An emulator is a program that simulates a CPU,programmers use assembly language – human-readable mnemonics for binary opcodes. An assembler translates assembly (e.g., ADD 4,8,12) into the actual machine code bits. A Virtual Machine uses virtualization to run multiple operating systems on one physical hardware. Extract instruction field: field = (inst >> k) and ((1 << width) - 1)shift right k bits, then mask to keep 'width' bits. VM advantages: run different/old OS; isolation from bugs/malware; test/dev many environments on one machine; consolidate many small systems on one big server (cloud).

Type	**Summary**
Emulator	Imitates a completely different system (e.g., different CPU or old console); full hardware simulation; slowest but most flexible.
Virtual Machine	Runs multiple OS instances on the same physical machine; each VM behaves like a full computer; requires a hypervisor with some performance overhead; strong OS-level isolation – different kernels possible.
Container	Shares the host OS kernel; isolates at application level (processes, file system, network per container); very low overhead but all containers must be compatible with host kernel; weaker isolation than VMs but usually efficient enough.

Docker is a lightweight container runtime sitting above the host OS (sharing its kernel), providing fast isolated environments that package apps with their dependencies and move easily between machines.

Processes and File Systems:: A program is the static code (on disk), while a process is that code in execution (in memory with resources),When run a program, pass arguments: $0 is the script name, $1 the first argument, $2 second, and $# the number of arguments.Processes execute one by one in the foreground; background processes can run in parallel, requiring a job table and jobs/fg/bg for management.

Shell runs commands with a fork–exec–wait pattern: the shell calls fork() to create a child process, the child calls exec() to replace itself with the target program (e.g. ls), and the parent just calls wait() to block until that child finishes, then returns to the prompt.Python: os.fork() → code after fork runs in both parent and child (pid==0 child); os.exec*() in child replaces its code, so lines after exec do not run if exec succeeds. IPC: processes communicate via pipes/FIFOs and special files; in Unix "everything is a file" (/dev/*, pipes, /dev/null).

FS impl: FS (ext4, NTFS, etc.) uses inodes (metadata + block pointers) and data blocks. Directories map name→inode; path resolution walks /dir/subdir/file via directories. Permissions: rwx for owner/group/others + UID/GID. ext4 = Linux FS; NTFS/FAT32/exFAT = Windows; APFS = macOS.

Symbolic links: symlink is a special file pointing to another path (like shortcut), created with 'ln -s target linkname'; has its own inode and stores a pathname.

Special files: devices (/dev/sda, /dev/tty), /proc, pipes, /dev/urandom, etc. all exposed as files so programs just open/read/write bytes.

Memory Management, Scheduling, Bootstrap, and Shell Scripting:

Text (Code)	Compiled instructions of the program.
Data	Global + static variables (initialized + BSS).
Heap	Dynamic memory from `malloc/new`; grows upward.
Stack	Function call frames + local variables; grows downward.
Layout	Each process has its own address space: stack grows down from high addresses, heap grows up from data segment; excessive recursion or heap growth may collide.

Base/limit mapping: each process sees virtual addresses from 0; hardware adds base and checks offset < limit. Physical = base + virtual; prevents a process from addressing memory outside its allocated range. Address bits = $\log_2 n$(memory bytes); 256B → 8 bits.

Virtual memory and paging: RAM split into fixed-size frames, process memory into pages. Per-process page table maps virtual page → frame or disk (swap). Gives isolation and allows non-contiguous allocation.

Demand paging and page faults: if a referenced page is on disk, hardware triggers a page fault; kernel loads the page from swap into a free frame, updates the page table, then resumes the process. This lets programs use more memory than physical RAM.

Page replacement and swapping: if no free frame, OS swaps out some page to disk to free a frame. Too frequent swapping to thrashing (system spends most time doing disk I/O). Use replacement algorithms (e.g. LRU) to keep hot pages in RAM.

Shared memory: some pages are mapped into multiple processes (e.g. shared code pages, or explicit shared-memory segments). Saves RAM and allows fast IPC.

Memory protection: a process can only access pages present in its page table, and only the kernel can modify the page table. PTEs carry R/W/X bits; OS can mark code pages read-only and data pages non-executable to prevent illegal access and some attacks.

Scheduler & states: ready → running (chosen by scheduler); running → blocked (I/O); running → ready (time slice expires, preempted).

CPU sharing: (improved) round-robin, each ready process gets a time quantum, giving illusion of parallelism on one CPU.

Foreground vs background: background job = process not using terminal; 'cmd &' runs in background; 'jobs', 'fg', 'bg' manage them.

Niceness / priority (Unix)

niceness ∈ [−20, +19], default 0. Lower nice ⇒ higher priority (more CPU). Normal users can only increase nice (be more "polite"), while decreasing nice (a negative value) usually requires root access.

Command	**Effect / priority**
`nice -n 10 cmd`	start `cmd` with nice=10 (lower priority than default)
`nice -n -5 cmd`	start `cmd` with nice=-5 (higher priority, root needed)
`renice -n 5 -p PID`	set process `PID` to nice=5 (lower priority / more polite)
`renice -n -10 -p PID`	set `PID` to nice=-10 (higher priority, decrease nice)

Bootstrapping	**What it does / key points**
BIOS / UEFI	Firmware in ROM/flash, runs first on power-on. Initialises basic hardware, reads boot order, selects boot device, then loads first-stage bootloader (e.g. from disk MBR).
MBR & first-stage bootloader	MBR = first 512 B of boot disk. Contains a tiny loader whose only job is to load a more capable second-stage bootloader (commonly **GRUB**).
GRUB (2nd-stage bootloader)	Understands filesystems, shows boot menu, then loads chosen OS kernel (e.g. `/boot/vmlinuz`) and optional initrd into RAM, and jumps to kernel entry point.
Kernel initialization	Kernel code starts. Sets up core subsystems (memory, scheduler, drivers), mounts root filesystem, then starts first user-space process (PID 1: `init` or `systemd`).
init / system processes	`init/systemd` starts services and daemons, configures system/network, then spawns login program or GUI. System reaches normal running state.

Handling command-line arguments	**Meaning**
`$0`	Script name.
`$1 ... $N`	Positional args 1..N.
`$#`	Number of command-line arguments.
`$@`	All arguments, each as separate word.
`[ $# -lt 2 ]`	Test: "number of args is less than 2".
`exit 0`	Normal success exit status.
`exit 1`	Error / failure (non-zero status).
`echo "msg" >&2`	Print `msg` to **stderr** (error output).

Loops & redirection	**Meaning**
`for x in list; do ...; done`	Shell for over items in `list`.
`for (( i=1; i<=N; i++ ))`	C-style numeric loop (Bash extension).
`> output.txt`	Redirect stdout, **overwrite** `output.txt`.
`>> output.txt`	Redirect stdout, **append** to `output.txt`.
`2> errors.log`	Redirect stderr (fd 2) to `errors.log`.

Conditions & file tests	**Meaning**
`[ ... ] / [[ ... ]]`	Test expression in `if`, `while` etc.
`-e file`	True if file exists.
`-d file`	True if file is a directory.
`-f file`	True if file is a regular file.
`-n "$s"`	True if string `$s` is non-empty.
`! cond`	Logical NOT (invert condition).

Arithmetic	**Meaning**
`(( expr ))`	Evaluate integer expression (no `expr` command needed).
`(( i++ ))`, `(( i += 1 ))`	Increment variable `i`.
`[ "$a" -lt "$b" ]`	Numeric: `a < b` (**less than**).
`[ "$a" -gt "$b" ]`	Numeric: `a > b` (**greater than**).
`[ "$a" -eq "$b" ]`	Numeric: `a == b` (**equal**).

Error handling & shebang	**Meaning**
`echo "Error: ..." >&2; exit 1`	Standard pattern: print error to stderr and exit with failure.
`#!/bin/bash`	Shebang: choose Bash as script interpreter.
`chmod +x script.sh`	Make script executable.
`./script.sh args`	Run script via shebang (must be executable).
`bash script.sh args`	Run script explicitly with Bash (no +x needed).

Introduction to Networks and Switching:

Network = collection of interconnected nodes (hosts, routers, switches) that share a medium and communicate using protocols.

Early designs copied the telephone system (circuit switching), but the Internet uses packet switching for efficiency and robustness.

Circuit vs packet switching	**Circuit switching**	**Packet switching**
Connection	Dedicated end-to-end circuit set up before data; path fixed for duration	No pre-set circuit; data split into packets, each routed hop-by-hop
Bandwidth	Reserved for one call (guaranteed)	Shared statistically among many flows
Delay	Predictable, constant once set up	Variable; depends on queueing, route choice, load
Failure	Link failure drops the call (no auto reroute)	Packets can be rerouted around failed/slow links
Efficiency	Wastes capacity if circuit idle	Good for bursty data traffic

Store-and-forward: each router receives a whole packet then forwards. One hop transmit delay ≈ L/R (packet size L bits, link rate R bps).

LANs, WANs, Ethernet, WiFi	**LAN**	**WAN**
Scope	Local area (home, office, campus)	Large area (cities, countries); interconnects many LANs
Speed & tech	High speed; Ethernet, WiFi	Often lower speed; ISP/backbone links
Ownership	Typically one organisation	Many organisations/ISPs
Example	Home/campus network	The global Internet (WAN of many networks)

Term	**Meaning**
Ethernet	Dominant wired LAN (link layer). Packet-switched; sends *frames* with src/dst MAC addresses.
WiFi	Wireless LAN technology (802.11). Similar framing idea to Ethernet but uses radio instead of cable.
MAC address	48-bit link-layer address (often written as 6 hex bytes) that uniquely identifies a NIC on a LAN. Used only locally.
Frame	Link-layer unit: header (src/dst MAC, type, CRC) + payload (e.g. IP packet). At each hop, the IP packet is encapsulated in that link's frame.

Internet Protocol provides best-effort delivery of **IP datagrams** across networks.

Item	**Key facts**
IP packet (datagram)	Header (src/dst IP, length, TTL, protocol, header checksum, etc.) + payload (TCP/UDP segment).
IPv4 address	32-bit value, written as dotted decimal (e.g. `129.78.8.1`). Identifies a host/interface in the global network.
IPv6 address	128-bit value, written in hex; not examined in detail in this course.
Routing	Routers forward IP packets towards destination IP using a routing table (prefix → next hop).
Error detection	IP header checksum; Ethernet adds CRC at link layer. Corrupted packets are detected and dropped (higher layers may retransmit).
ICMP	Internet Control Message Protocol: diagnostic/control messages (e.g. `ping` uses Echo Request/Reply).

UDP vs TCP	**UDP**	**TCP**
Type	Connectionless, best-effort datagrams	Connection-oriented, reliable byte stream
Reliability	No guarantees: packets may be lost, duplicated, reordered	Reliable: seq/ack, retransmission, reordering, flow and congestion control
Overhead	Low header overhead, no connection setup	Higher overhead; 3-way handshake and per-connection state
Typical uses	Simple query/response or streaming where loss is tolerable (e.g. DNS lookups on port 53, some media streaming)	Most applications needing integrity and ordering (HTTP, HTTPS, SMTP, SSH, etc.)

Ports: 16-bit numbers (0–65535) identifying application endpoints on a host (e.g. 80 HTTP, 22 SSH, 53 DNS).

Layer	**OSI name**	**TCP/IP model**	**Examples**
7	Application	Application	HTTP, FTP, DNS, SMTP, SSH
6	Presentation	(merged into Application)	Encryption, compression, data formats
5	Session		Sessions, dialogs, checkpoints
4	Transport	Transport	TCP, UDP
3	Network	Network	IP, ICMP, routing
2	Data Link	Link / Network access	Ethernet, WiFi, PPP
1	Physical		Bits on wire/fibre/radio

OSI 7-layer model is mostly theoretical; the Internet TCP/IP stack is usually shown as 5 layers: Application, Transport, Network, Link, Physical.

Encapsulation: each layer adds its own header (e.g. HTTP data → TCP segment → IP packet → Ethernet frame).

Local networking	Role
ARP (Address Resolution Protocol)	On a LAN, maps IP addresses to MAC addresses. Host A broadcasts "Who has 10.0.0.7?", host B replies with its MAC; A then sends the IP packet inside an Ethernet frame to B's MAC. Used only for local delivery.
Switch	Layer-2 (Ethernet) device. Learns MAC addresses and forwards frames only on the port where the destination MAC is connected. Used to build efficient LANs.
Router	Layer-3 (IP) device. Uses routing table to forward IP packets between different networks. A home router connects your LAN (e.g. 192.168.1.0/24) to the ISP and usually performs NAT.

The Internet Protocol and Routing: **IP packets (IPv4)**
Network layer (IP) moves packets across multiple networks. Each IP packet has a header + data. Key fields: source IP, destination IP, TTL (time-to-live, avoid loops), protocol (TCP/UDP/ICMP), header checksum, plus length, flags, etc.
Max IPv4 packet size: 65 535 bytes (16-bit length field). Links have smaller MTUs, so large IP packets may be *fragmented* into smaller ones.

Feature	TCP	UDP	ICMP
Type	Connection-oriented, reliable byte stream	Connectionless, unreliable datagrams	Control / error messages over IP
Reliability	Seq/ack, retransmit, ordering, flow & congestion control	No reliability or ordering	Indicates errors or status (no user data)
Typical use	Web (HTTP/HTTPS), email (SMTP), SSH, etc.	DNS queries, some streaming/VoIP	ping (Echo Req/Reply), Time Exceeded, etc.
Layer	Transport	Transport	"Just above" IP (Network layer helper)

Transport ports:Ports distinguish services on a host. Ports are 16-bit (0–65535). Ports < 1024 are *well-known*. Examples:

Service	Port	Protocol
DNS lookup	53	UDP (sometimes TCP for large zone transfers)
HTTP	80	TCP
HTTPS	443	TCP
SMTP email	25	TCP
SSH	22	TCP

Each router examines destination IP and forwards according to its **routing table**. Entries look like: *"for network X use next-hop Y via interface Z"*. Routers choose the *longest prefix match* (most specific network).
Examples of entries:

- `10.0.2.0/24 via 10.0.5.1` (means any IP 10.0.2.* goes to router 10.0.5.1)
- `0.0.0.0/0 via 10.0.5.254` (default route for everything else)

Hosts have an IP and a netmask:

- If dest IP is in the same subnet (mask match), host uses ARP and sends directly.
- Otherwise it sends the packet to its **gateway** (router), which forwards hop-by-hop.

ARP, traceroute & tools	Role
ARP	Address Resolution Protocol (IPv4). On a LAN, map IP → MAC: broadcast "Who has IP X?", owner replies with its MAC. Sender caches IP→MAC in an ARP table.
Neighbor Discovery (ND)	IPv6 equivalent of ARP (uses ICMPv6).
traceroute / tracert	Shows path (router hops) to a destination. Sends packets (UDP or ICMP) with increasing TTL; routers send ICMP "Time Exceeded" when TTL hits 0.
ping	Uses ICMP Echo Request/Reply to test reachability and measure RTT.
ipconfig/ifconfig, route/ip route	Show IP configuration and routing table on a host (Windows/Unix).

NAT & private addresses:
Because IPv4 addresses are limited, many networks use **private address ranges** (not routed on the public Internet):

Private range	Notes	
10.0.0.0 – 10.255.255.255	Class A private range	
192.168.0.0 – 192.168.255.255	Common home/office LAN range (Class C).	

(A third block 172.16.0.0–172.31.255.255 also exists but is less commonly seen in small LANs.)
A NAT router translates internal private IPs to a shared public IP: rewrites outgoing packets (source IP & port) and tracks the mapping so replies can be delivered back to the correct internal host. NAT extends IPv4's life but breaks direct incoming connections (unless port forwarding is configured).

Property	IPv4	IPv6
Address size	32 bits (4 bytes)	128 bits (16 bytes)
Notation	Dotted decimal (e.g. 129.78.8.1)	Hex groups separated by ":" (e.g. 2001:0db8:85a3::8a2e:0370:7334)
Space	∼4.3 billion addresses	Vastly larger; essentially inexhaustible
Private ranges / NAT	Uses private blocks + NAT to stretch address space	Space so large that NAT mostly unnecessary
Address discovery	ARP	Neighbor Discovery (ND, ICMPv6), no ARP

IPv6 simplifies some aspects (no ARP, routers do not fragment packets; hosts should use Path MTU discovery) and has mandatory support for IPsec. Adoption is ongoing but IPv4+NAT is still very common.

DNS (Domain Name System)
Distributed, hierarchical database that maps human-readable domain names (e.g. `www.sydney.edu.au`) to IP addresses (and sometimes IP → name). Queries usually use UDP port 53.

DNS concepts	Key idea
Hierarchy	Names are hierarchical and read right-to-left: `host.subdomain.domain.TLD`. Root "." at top, then TLDs (.com, .au, . . .), then organisation (e.g. `sydney`), then host (e.g. `www`).
Zones & authority	Namespace split into zones. Each zone is managed by an authority (organisation) and served by one or more authoritative name servers.
Resolvers	A client typically asks a local *recursive resolver* (provided by ISP/uni). Resolver checks its cache; if miss, it walks the tree: root → TLD → . . . → authoritative server, then returns the final answer.
Protocols	DNS messages are usually sent using UDP port 53. For large responses or special operations, TCP may be used.
DHCP & DNS	When a host joins a network, DHCP (over UDP) can assign it an IP address, netmask, default gateway, *and* which DNS server to use.
Caching & TTL	Every record has a TTL (Time To Live). Resolvers cache answers until TTL expires, reducing load but meaning DNS changes take time to propagate.

Common DNS record types	Meaning
A	IPv4 address record: name → IPv4 address.
AAAA	IPv6 address record: name → IPv6 address.
MX	Mail exchanger: which mail server(s) handle email for this domain.
CNAME	Canonical name (alias): one name refers to another DNS name.
NS	Name server record: which servers are authoritative for this zone.
PTR	Reverse lookup: IP → name (stored under special reverse zones).
TXT	Free-form text info (SPF, DKIM, misc metadata, etc.).

Application layer protocols
Application protocols run on top of TCP/UDP and define how data is structured between client and server (usually text-based commands and replies).

protocols	Proto	Port	Notes
HTTP	TCP	80	Web pages; request/response; stateless.
SMTP	TCP	25	Mail submission/relay (server↔server).
POP3	TCP	110	Client downloads mail (local store).
IMAP4	TCP	143	Online mail access; folders & sync on server.
FTP	TCP	20/21	Legacy file transfer (separate ctrl/data).
SSH	TCP	22	Secure remote shell; also scp/sftp tunnels.
DNS	UDP/TCP	53	Name resolution; UDP for queries, TCP for large/zone xfer.
DHCP	UDP	67/68	Auto-assign IP/mask/gateway/DNS; LAN broadcast.
HTTPS	TCP	443	HTTP over TLS; encrypted web traffic.

Browser needs IP for `www.sydney.edu.au`: OS asks the configured DNS resolver (UDP 53), which returns an A/AAAA record. Browser opens a TCP connection to that IP on port 80 (or 443 for HTTPS). Client sends an HTTP `GET /` request; server responds with status line, headers, and HTML content. Under the hood this path also uses: ARP / routing (Weeks 6–7), TCP reliability, DNS caching, NAT (if behind a home router), etc.
Cloud computing: On-demand access to shared computing resources (servers, storage, DB, networking, apps) over the Internet. Key traits: on-demand self-service, broad network access, resource pooling (multi-tenancy), rapid elasticity, measured service (pay-as-you-go).
Service models: IaaS = VMs + storage + networks, you manage OS/apps (e.g. AWS EC2). PaaS = provider manages runtime/DB, you deploy code (e.g. App Engine, Azure App Service).
SaaS = full application delivered over web (e.g. Gmail, Office 365).
Typical AWS-style services: Compute: EC2 (VMs), Lambda (serverless). Storage: S3 (object), EBS (block), Glacier (archive). DB: RDS (managed SQL), DynamoDB (NoSQL). Networking: VPC, Route 53 (DNS), Load Balancer, API Gateway. Security: IAM (identity/roles), KMS (keys).
Auto-scaling: Use metrics (CPU, requests/s) and cloud APIs to automatically add/remove instances behind a load balancer. Only pay for extra capacity when needed.
Serverless (Functions-as-a-Service): e.g. AWS Lambda. You upload a function; platform provisions containers/VMs on demand, runs code, then tears them down. Billed per invocation and execution time, scales automatically.
IoT & Edge: IoT devices send data to the cloud for storage and processing. Edge/CDN nodes place compute/cache closer to users/devices to reduce latency and backbone traffic (e.g. CloudFront).
Microservices: Break large apps into many small services communicating over the network (often HTTP+JSON / gRPC). Each service can be independently deployed and scaled; fits well with containers and serverless.
Benefits of cloud (exam-style): lower upfront hardware cost, elastic capacity, global deployment, managed services (DB, security, monitoring). **Risks**: provider outages, security/misconfiguration, vendor lock-in, dependence on reliable Internet.
Security goals: Confidentiality (only authorised can read); Integrity (changes are detectable); Authenticity (know who sent it); Non-repudiation (sender cannot credibly deny).
Basics: Encryption transforms plaintext m to ciphertext $c = E_k(m)$, decryption $m = D_k(c)$. Key k is secret; without k, c should reveal essentially nothing about m.
Symmetric (secret-key) crypto: Same key for encryption and decryption (shared by Alice & Bob). Fast, used for bulk data (e.g. AES; DES is legacy). Main issue: key distribution (how to share the key securely).
Asymmetric (public-key) crypto: Key pair $(k_{\text{pub}}, k_{\text{priv}})$; encrypt with public key, decrypt with private. Solves key distribution: publish k_{pub}, keep k_{priv} secret. RSA is a classic *asymmetric* algorithm (not symmetric), used for small data (e.g. encrypting session keys, digital signatures). Diffie–Hellman / ECDH: protocol to agree on a shared secret key over an insecure channel.
Digital signatures: Provide authenticity, integrity, non-repudiation. Sender computes $h = \text{hash}(m)$ then signature $s = \text{Sign}_{k_{\text{priv}}}(h)$. Verifier recomputes $h' = \text{hash}(m)$ and checks $\text{Verify}_{k_{\text{pub}}}(h', s)$. Only the holder of k_{priv} can create a valid signature.
Hash functions: Map arbitrary-length input to fixed-length digest; one-way and collision-resistant. Used for integrity checks, password storage (store hash, not password), and inside signatures. Common exam sizes: SHA-256 ⇒ 256 bit = 32 bytes = 64 hex digits; SHA-384 ⇒ 384 bit = 48 bytes = 96 hex digits.
TLS / HTTPS: TLS runs between Application and TCP. Browser connects to server, verifies CA-signed certificate (public-key + signatures), performs key exchange (DH/RSA) to obtain a symmetric session key, then encrypts HTTP traffic (confidentiality + integrity). `https://` = HTTP over TLS.
Other secure uses: SSH / SCP: secure remote login and file copy using similar public-key+symmetric combo. PGP: email encryption and signatures using public keys and a "web of trust". Code signing: binaries/updates are signed; clients verify with vendor public key. Blockchains: transactions authorised by digital signatures (concept only).
Performance Analysis:
Core ideas: Performance matters for latency, throughput, scalability, reliability, and cost. We look for bottlenecks (slowest stage) and high utilization components.
Key metrics: Latency = time per request (e.g. ms). Throughput = requests / sec (or bits / sec). Utilization U = fraction of time resource is busy (CPU, disk, link). Scalability: how performance changes when we add more servers. Queueing delay: extra waiting time when demand $\approx$ or $>$ service rate.
Packet delay (per hop): Total nodal delay

$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

Processing d_{proc} (check header, bit errors; usually tiny). Queueing d_{queue} (waiting in buffer; depends heavily on load). Transmission $d_{\text{trans}} = L/R$ (packet length L bits, link rate R bps). Propagation $d_{\text{prop}} = d/s$ (distance d, propagation speed s).
Traffic intensity & queueing: Let arrival rate a (pkts/s), packet size L (bits), service rate R/L. Traffic intensity $\rho = La/R$. If $\rho \ll 1$: queueing small; $\rho \to 1$: delay ↑ sharply; $\rho > 1$: unstable (infinite backlog). Simple M/M/1 model: average queue length $N = \rho/(1-\rho)$, average delay $D = 1/((R/L) - a)$.
Throughput & bottlenecks: Along an end-to-end path, average throughput is limited by the bottleneck rate:

$$T_{\text{end-to-end}} = \min(R_1, R_2, \dots)$$

If n flows fairly share a bottleneck of rate R, each gets about R/n. Throughput vs latency trade-off: running a link at very high utilization gives good throughput but large queueing delay.
Little's Law (intuitive form): $L = \lambda W$: average number in system L = arrival rate $\lambda \times$ average time W spent in system. Useful for relating queue length, throughput, and delay.
Amdahl's Law (high level): If fraction f of work can be sped up by factor k, overall speedup

$$S = \frac{1}{(1-f) + f/k}$$

Even huge k gives limited benefit when serial part $(1-f)$ is large focus on big bottlenecks.
Means for performance data: Arithmetic mean: simple average of times; can hide skewed workloads. Weighted mean: $\sum p_i t_i$ using execution frequencies p_i. Geometric mean: $(\prod r_i)^{1/n}$, good for comparing relative speed ratios across many programs. Harmonic mean: $H = n/\sum(1/x_i)$, appropriate when x_i are rates (throughputs); emphasizes slowest rates.
Measurement & benchmarking: Measure, don't guess: use profilers, CPU / disk / network counters, ping/traceroute, load tests. Be careful of pitfalls: cherry-picked stats, quoting only peak numbers, "apples vs oranges" benchmarks. No single benchmark captures all performance; use multiple workloads and look at both speed and cost.